

Universidad
Rey Juan Carlos

ESCUELA TÉCNICA SUPERIOR
DE INGENIERÍA INFORMÁTICA

GRADO EN INTELIGENCIA ARTIFICIAL

Práctica Obligatoria

**Procesamiento del Lenguaje
Natural**

Curso 2025-2026

Autor: Yasin Hanin Boukaddidi

1. Introducción.....	3
1.1. Problema y objetivo.....	3
1.2 Dataset.....	3
1.2.1 Distribución de las clases.....	3
1.2.2 Distribución del tamaño de los datos.....	4
2. Modelos.....	4
2.1. Preprocesamiento.....	4
2.2. Propuesta 1: Modelos clásicos con BoW.....	5
Vectorización (pesado binario, TF-IDF).....	5
Pesado binario.....	5
TF-IDF.....	6
Modelos: Naive Bayes, Regresión Logística, SVM.....	6
Experimentos y resultados.....	7
Análisis.....	8
2.3. Propuesta 2: Modelos clásicos con embeddings estáticos.....	8
FastText/BioWordVec.....	8
Modelos clásicos.....	9
Experimentos y resultados.....	9
Análisis.....	9
2.4. Propuesta 3: Deep Learning con embeddings estáticos.....	10
Arquitectura Sin BiLSTM / Con BiLSTM.....	10
Experimentos y resultados.....	10
Análisis.....	11
2.5. Propuesta 4: Embeddings contextuales.....	12
Clinical Bert.....	12
Arquitectura Sin BiLSTM vs Con BiLSTM.....	12
Sin BiLSTM.....	12
BiLSTM.....	13
Experimentos y resultados.....	13
Análisis.....	14
2.6. Propuesta 5: Fine-tuning.....	15
ClinicalBERT.....	15
Experimentos y resultados.....	16
Análisis.....	17
2.7. Modelo Final y predicciones.....	17
3. Conclusiones.....	17
Referencias.....	19

1. Introducción

1.1. Problema y objetivo

El contexto del problema es uno médico, concretamente la identificación de la estadificación del cáncer de un paciente a partir de un diagnóstico.

Proporcionado un diagnóstico de un médico sobre un paciente en lenguaje natural, nuestro objetivo es determinar en qué estadificación se encuentra. En nuestro problema, los casos se reducen a 4, es decir, es un problema de clasificación.

La forma de hacerlo será con un modelo de PLN. Y para ello se desarrollaron diversas propuestas con distintos enfoques que explicaré. Pero previamente haremos un análisis del dataset proporcionado para la práctica, ya que dependiendo de las características un enfoque podría ser más ideal que otro.

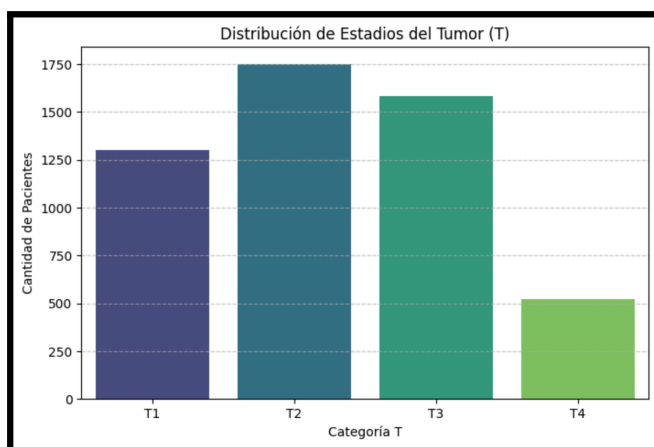
1.2 Dataset

El dataset proporcionado contiene 5158 datos sobre las 4 clases, cuyo lenguaje usado es inglés, con anotaciones médicas.

En la entrega se proporciona un notebook,

1.2.1 Distribución de las clases

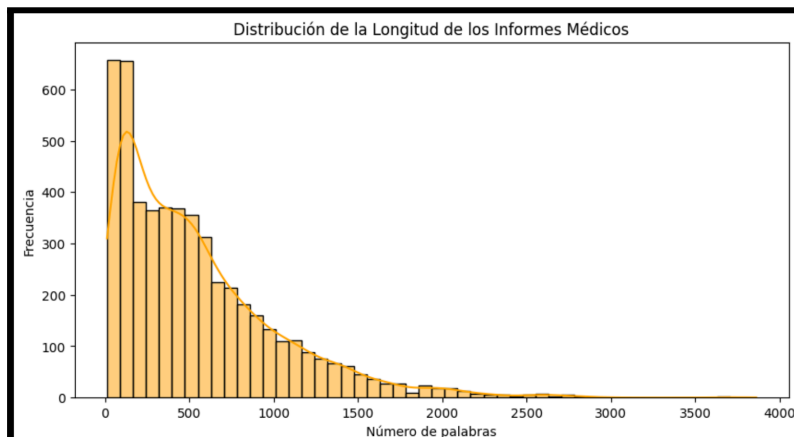
Como se puede ver a continuación, las clases se distribuyen decentemente a excepción de la clase T4:



Las clases T1, T2, y T3 están entre 1250 y 1750, pero T4 apenas llega a 500. Esto es algo que puede ser problemático, ya que nos puede ser insuficientes datos de la clase a la hora de entrenar. Una técnica que se considera es Data Augmentation.

1.2.2 Distribución del tamaño de los datos

También podemos observar abajo la distribución de la longitud, en palabras, de los textos:



Vemos que se da una distribución LogNormal que llega hasta las 3500 palabras y que se distribuye mayoritariamente en 0 y 500. Esto es importante saberlo ya que podemos usar modelos, que como es el caso, tiene un límite máximo de 512 tokens. Esto lo solucionamos más adelante cogiendo una ventana de 430 (la mediana) y adaptando el resto de texto con técnicas como padding y truncation.

2. Modelos

En este apartado vamos a explicar los modelos usados, así como la arquitectura, técnicas, resultados, etc.

En la entrega se proporciona un notebook por cada propuesta. Es decir, para la propuesta 1 se da Propuesta1.ipynb, propuesta 2 se da Propuesta2.ipynb, para la propuesta 3 se da Propuesta3.ipynb, para la propuesta 4 se da Propuesta4.ipynb, propuesta 5 se da Propuesta5.ipynb.

Las propuestas 3, 4 y 5 requieren uso de GPU y han sido desarrolladas desde Google Colab.

2.1. Preprocesamiento

Un preprocesamiento de los datos es necesario en la mayoría de los modelos que han sido desarrollados. El procedimiento fue el siguiente:

*importamos un tokenizador → realizamos la tokenización del texto → nos quedamos solo con los tokens relevantes**

* (que no sean signos de puntuación, que no sean palabras vacías, que sean alfanuméricos, etc.)

Así reducimos la cantidad de palabras a analizar, quitándonos de encima aquellas que no nos proporcionan información relevante.

Además, como se explicará en el siguiente apartado, de cada token sacaremos un embedding de cada token, los cuales guardaremos en una matriz cuyo tamaño depende directamente de la cantidad de tokens que tengamos (asumiendo que están en su vocabulario).

Los tokenizadores que se han usado son de Spacy[1] y de modelos preentrenados Bert.

Un problema de los tokenizadores de modelos pre-entrenados como los de Bert es que cuando no reconocen una palabra la dividen en sub-tokens que sí reconoce.

Esto se puede solucionar uniendo el primer sub-token con los que siguen mientras comience con ‘##’, notación que se usa para indicar la división de una palabra.

2.2. Propuesta 1: Modelos clásicos con BoW

Como primera aproximación al problema se decidió utilizar representaciones basadas en **Bag of Words (BoW)** junto con modelos clásicos de aprendizaje automático. La principal razón es que este tipo de enfoques son relativamente sencillos de implementar, no muy costosos computacionalmente y que suelen servir como un buen punto de partida en tareas de clasificación de texto antes de probar modelos más complejos.

La idea detrás de esta propuesta es que ciertas palabras presentes en los diagnósticos médicos pueden aportar información relevante sobre la estadificación del cáncer. Por ejemplo, determinados síntomas, procedimientos o términos clínicos podrían aparecer con más frecuencia en unas etapas que en otras, ayudando así al modelo a distinguir entre clases.

Vectorización (pesado binario, TF-IDF)

Dado que los modelos clásicos de aprendizaje automático no pueden operar directamente sobre texto, fue necesario transformar los diagnósticos en representaciones numéricas mediante técnicas de vectorización basadas en Bag of Words.

Pesado binario

La primera representación empleada fue un esquema de **pesado binario**, en el que cada documento se representa mediante un vector donde cada dimensión corresponde a un término

del vocabulario. El valor asociado toma valor 1 si el término aparece en el documento y 0 en caso contrario.

Este enfoque resulta adecuado bajo la hipótesis de que la mera presencia de determinados términos clínicos puede ser suficiente para identificar una estadificación específica, independientemente de su frecuencia dentro del texto.

TF-IDF

Adicionalmente, se utilizó la representación **TF-IDF (Term Frequency–Inverse Document Frequency)**, que pondera cada término según su frecuencia en el documento y su rareza dentro del corpus completo.

El uso de TF-IDF se justifica porque, en el contexto médico, no solo puede ser relevante la presencia de determinados conceptos clínicos, sino también su importancia relativa dentro del diagnóstico. Asimismo, esta representación reduce el peso de términos muy frecuentes y poco discriminativos, favoreciendo aquellos más específicos del dominio clínico.

Modelos: Naive Bayes, Regresión Logística, SVM

Una vez obtenidas las representaciones vectoriales mediante Bag of Words, se probaron distintos modelos clásicos de clasificación para comprobar cuál se adaptaba mejor al problema. Los modelos escogidos fueron **Naive Bayes**, **Regresión Logística** y **Support Vector Machine (SVM)**, ya que son algoritmos muy usados en clasificación de texto y suelen funcionar bien con representaciones de alta dimensionalidad como BoW o TF-IDF.

Naive Bayes

El primer modelo utilizado fue **Multinomial Naive Bayes** [6], un algoritmo probabilístico ampliamente utilizado en PLN por su simplicidad y rapidez de entrenamiento.

Este modelo calcula la probabilidad de que un texto pertenezca a una clase concreta a partir de las palabras presentes en él. Aunque asume independencia entre palabras (algo que realmente no se cumple en lenguaje natural), suele dar resultados razonables en tareas de clasificación textual y sirve como una buena referencia inicial para comparar otros enfoques más complejos.

Regresión Logística

También se utilizó **Regresión Logística** [5], un modelo de clasificación supervisada bastante común en problemas de texto.

A diferencia de Naive Bayes, no asume independencia entre las características y aprende pesos para cada término del vocabulario, permitiendo modelar mejor la relación entre palabras y clases. Este tipo de modelo suele comportarse bien cuando se combina con

representaciones TF-IDF, por lo que resultaba interesante comprobar su rendimiento en el contexto de diagnósticos médicos.

Support Vector Machine (SVM)

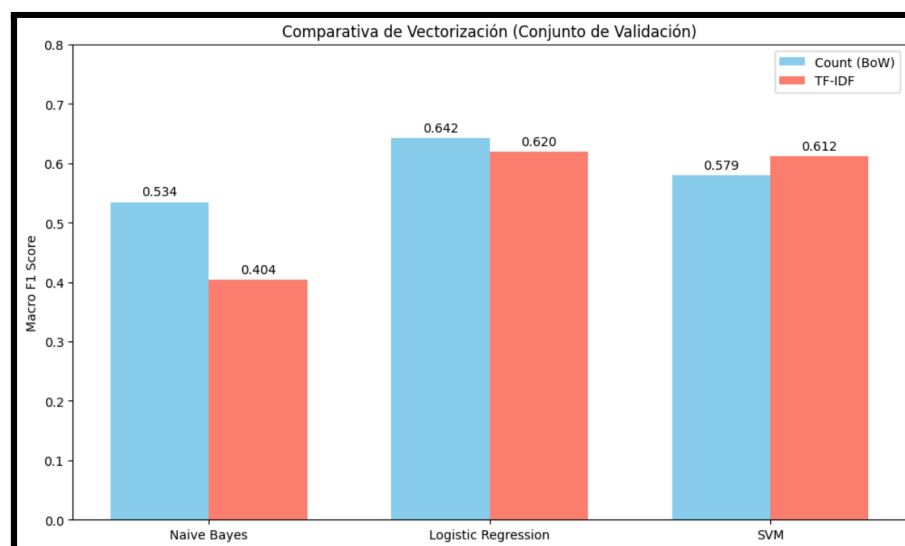
Por último, se empleó **SVM** [4], un modelo clásico que históricamente ha obtenido buenos resultados en tareas de clasificación de texto.

La idea principal de SVM es encontrar una frontera de separación entre clases que maximice la distancia entre ejemplos. Debido al carácter técnico y específico del lenguaje médico, se esperaba que SVM pudiera capturar mejor ciertas diferencias entre diagnósticos de distintas estadificaciones.

Experimentos y resultados

Antes de entrenar los modelos, se realizó un preprocesamiento de los textos utilizando el modelo de SpaCy **en_core_sci_lg** [1], especializado en terminología científica y biomédica en inglés. Sobre los diagnósticos se aplicó el proceso descrito anteriormente: tokenización, eliminación de elementos poco informativos (signos de puntuación, *stopwords* y tokens no alfanuméricos) y lematización.

Una vez preprocesados los datos, se generaron dos tipos de representaciones textuales: **pesado binario** con CountVectorizer [2] y **TF-IDF** con TfidfVectorizer [3]. Sobre cada una de ellas se entrenaron los tres modelos clásicos planteados previamente (**Naive Bayes**, **Regresión Logística** y **SVM**) con el objetivo de comparar tanto el efecto de la vectorización como del clasificador empleado.



Análisis

A partir de los resultados obtenidos, se puede ver que **Regresión Logística con pesado binario** fue el modelo que mejor funcionó, alcanzando un **Macro F1 de 0.6425**. En general, Regresión Logística y SVM dieron mejores resultados que Naive Bayes.

Llama la atención que el **pesado binario funcionase mejor que TF-IDF** en el mejor modelo. Esto puede indicar que, para este problema, la presencia de ciertos términos médicos es más importante que el número de veces que aparecen en el diagnóstico.

Por otro lado, aunque los resultados son razonables para una primera aproximación, este enfoque tiene una limitación importante: **Bag of Words no tiene en cuenta el contexto ni el orden de las palabras**, algo especialmente relevante en textos médicos. Por ello, en las siguientes propuestas se exploran modelos basados en embeddings.

2.3. Propuesta 2: Modelos clásicos con embeddings estáticos

En esta segunda propuesta se utilizan **embeddings estáticos** como representación del texto, en lugar de representaciones basadas en Bag of Words. La idea es que estos embeddings permiten capturar cierta información semántica de las palabras, lo que puede ser útil en un dominio como el médico, donde existen términos poco frecuentes o con significados muy específicos.

FastText/BioWordVec

En concreto, se ha utilizado **FastText a través de los embeddings preentrenados BioWordVec** [8], descargados directamente desde su repositorio oficial. Estos embeddings están entrenados sobre textos biomédicos, por lo que resultan adecuados para este problema.

BioWordVec proporciona dos versiones principales: *intrinsic* y *extrinsic*. En este caso se ha utilizado la versión **extrinsic**, ya que, según la documentación, está mejor orientada a tareas de clasificación de texto. Esta variante utiliza una ventana de contexto más pequeña (5), mientras que la versión *extrinsic* utiliza una ventana mayor (20).

*“We created two specialized, task-dependent sets of word embeddings “Bio-embedding-intrinsic” and “Bio-embedding-extrinsic” via setting the context window size as 20 and 5, respectively. The pre-trained BioWordVec data are freely available on Figshare. “Bio-embedding-intrinsic” is for intrinsic tasks and used to calculate or predict semantic similarity between words, terms or sentences. **“Bio_embedding_extrinsic” is for extrinsic tasks and used as the input for various downstream NLP tasks, such as relation extraction or text classification.**”* [8]

El uso de embeddings pre-entrenados permite además manejar mejor palabras poco frecuentes o términos médicos específicos que no aparecen habitualmente en vocabularios generales.

Modelos clásicos

Sobre la representación obtenida a partir de los embeddings se han probado varios modelos clásicos de clasificación:

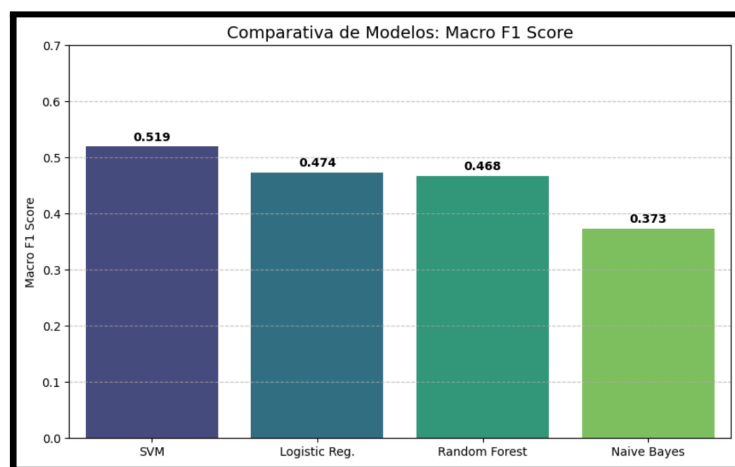
- **Regresión Logística**
- **SVM**
- **Random Forest**
- **Naive Bayes**

La idea es evaluar si una representación más rica a nivel semántico mejora el rendimiento respecto a las técnicas basadas en Bag of Words, manteniendo al mismo tiempo modelos de aprendizaje automático tradicionales.

Experimentos y resultados

Una vez obtenidos los embeddings de BioWordVec, se construyó un vocabulario a partir de los tokens generados en el preprocesamiento. Cada palabra se representó mediante su embedding correspondiente, permitiendo convertir los diagnósticos en vectores sobre los que entrenar los distintos modelos clásicos.

A continuación podemos ver una comparativa de los resultados:



Análisis

Los resultados fueron algo peores que en la propuesta anterior con **Bag of Words**. El mejor modelo fue **SVM lineal**, con un **Macro F1 de 0.5194**, aunque sin llegar a superar a Regresión Logística con pesado binario.

En general, la clase **T3 fue la que mejor se clasificó**, mientras que **T4 volvió a ser la más complicada**, probablemente por tener bastantes menos ejemplos en el dataset. Se probó con **Data Augmentation** para mejorar T4, pero en general, los resultados no fueron significativamente mejores.

Aunque se esperaba que **FastText** funcionase bien en un contexto médico por su capacidad para manejar palabras poco frecuentes, en este caso no aportó una mejora clara. Esto puede indicar que, para este problema, la presencia de ciertos términos concretos es más importante que la información semántica que aportan los embeddings.

2.4. Propuesta 3: Deep Learning con embeddings estáticos

En esta propuesta se pasa de modelos clásicos a **redes neuronales profundas**, aprovechando los embeddings obtenidos anteriormente. La idea es comprobar si modelos capaces de aprender relaciones más complejas entre palabras pueden mejorar los resultados obtenidos hasta el momento.

Se probaron dos arquitecturas distintas: una más sencilla, sin capas recurrentes, y otra incorporando una **BiLSTM**, con el objetivo de analizar si tener en cuenta el contexto secuencial de las palabras aporta una mejora.

Arquitectura Sin BiLSTM / Con BiLSTM

La primera arquitectura utilizada fue una versión sencilla compuesta por una capa de **Embedding**, seguida de una operación de **Pooling**, una capa de **Dropout** para reducir sobreajuste y una capa final **Dense** para realizar la clasificación.

Embedding → Pooling → Dropout → Dense

La segunda propuesta incorpora una capa **BiLSTM (Bidirectional LSTM)** antes de la clasificación:

Embedding → BiLSTM → Dropout → Dense

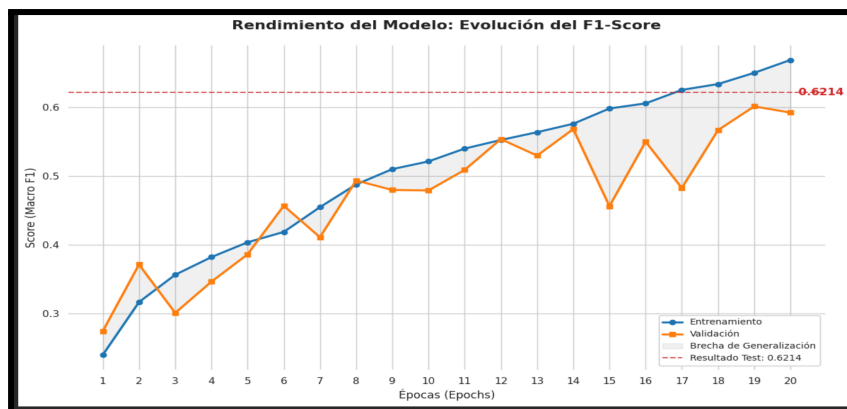
La principal diferencia es que la BiLSTM permite procesar la secuencia en ambas direcciones, pudiendo capturar mejor relaciones entre términos dentro del diagnóstico médico.

Experimentos y resultados

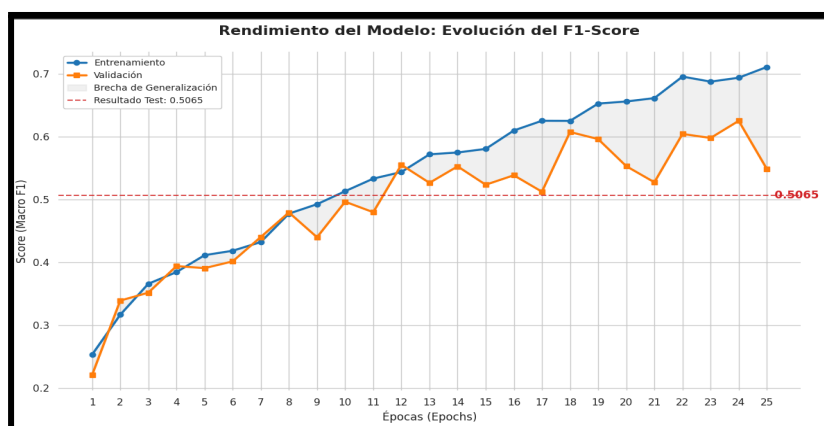
Para esta propuesta se utilizó el mismo vocabulario y los mismos embeddings de **BioWordVec** empleados en la propuesta anterior.

En ambos casos se utilizó el optimizador **Adam** y la función de pérdida **categorical cross entropy**, al tratarse de un problema de clasificación multiclase.

Sin BiLSTM:



Con BiLSTM:



Análisis

Los resultados fueron mejores que en la propuesta anterior con modelos clásicos, especialmente en la arquitectura **sin BiLSTM**, que alcanzó un **Macro F1 de 0.6256** en validación. Esto supone una mejora notable respecto al uso de embeddings estáticos con SVM o Regresión Logística.

Sin embargo, la arquitectura con **BiLSTM no consiguió mejorar los resultados**. Aunque el rendimiento en entrenamiento aumentó rápidamente (llegando a valores muy altos de *accuracy* y *Macro F1*), las métricas de validación empeoraron tras las primeras épocas. Esto indica un claro **sobreajuste (overfitting)**: el modelo aprendía muy bien los datos de entrenamiento, pero no conseguía generalizar correctamente a nuevos ejemplos.

Una posible explicación es el tamaño relativamente reducido del dataset para entrenar una arquitectura más compleja como una BiLSTM, además del desbalance entre clases. En este caso, una arquitectura más simple terminó ofreciendo mejores resultados.

Si quieres hacerlo todavía más visual, mete una gráfica de train vs validation loss; el overfitting se ve clarísimo y queda muy bien en memoria.

2.5. Propuesta 4: Embeddings contextuales

En esta propuesta se trabajó con **embeddings contextuales**, utilizando **ClinicalBERT**. A diferencia de FastText, donde cada palabra tiene siempre el mismo vector, en este caso la representación cambia según el contexto en el que aparece. Esto puede ser útil en textos médicos, donde muchos términos dependen bastante de cómo estén escritos dentro del diagnóstico.

Clinical Bert

Se utilizó **ClinicalBERT** [9], un modelo ya entrenado sobre textos clínicos y médicos. La idea de usarlo era aprovechar un modelo especializado en este tipo de lenguaje, ya que los diagnósticos contienen términos técnicos y abreviaturas que no suelen aparecer en lenguaje general.

Se decidió utilizar **ClinicalBERT** frente a **BioBERT** [11] debido a la naturaleza del dataset empleado. Mientras que **BioBERT** está entrenado principalmente sobre artículos biomédicos y literatura científica (por ejemplo, *PubMed*), **ClinicalBERT** ha sido entrenado sobre notas clínicas reales y documentación hospitalaria.

Dado que los textos de este proyecto consisten en **diagnósticos médicos redactados en lenguaje clínico**, con terminología técnica, abreviaturas y una estructura más cercana a registros médicos reales que a artículos científicos, se consideró que **ClinicalBERT podía adaptarse mejor al problema**.

Arquitectura Sin BiLSTM vs Con BiLSTM

Igual que en la propuesta anterior, se probaron dos arquitecturas: una más simple y otra añadiendo una capa **BiLSTM** para ver si ayudaba a capturar mejor el contexto del texto.

Sin BiLSTM

La primera arquitectura planteada fue una versión más sencilla, utilizando directamente los embeddings preentrenados como entrada del modelo.

Se partió de una capa de **Embedding**, inicializada con los embeddings obtenidos previamente y permitiendo su ajuste durante el entrenamiento (*fine-tuning*) para adaptarlos mejor al dominio médico.

En lugar de utilizar una capa recurrente, se aplicó una operación de **Global Average Pooling**, que resume la información de toda la secuencia en un único vector representativo del diagnóstico. Esta decisión se tomó para reducir el número de parámetros del modelo y disminuir el riesgo de sobreajuste.

Posteriormente, se añadieron capas de **Dropout** para mejorar la capacidad de generalización del modelo, junto con una capa **Dense** con activación **ReLU** antes de la clasificación final mediante **Softmax**.

La arquitectura utilizada fue:

Embedding → GlobalAveragePooling1D → Dropout → Dense(ReLU) → Dropout → Dense(Softmax)

BiLSTM

La arquitectura propuesta parte de una capa de **Embedding**, inicializada con los embeddings preentrenados obtenidos previamente. En este caso, se permitió que los embeddings siguieran ajustándose durante el entrenamiento (*fine-tuning*), con el objetivo de adaptarlos mejor al vocabulario y contexto específico de los diagnósticos médicos.

A continuación, se incorporó una capa **Bidirectional LSTM (BiLSTM)** con 32 unidades, permitiendo procesar la secuencia de palabras en ambas direcciones. La intención era capturar relaciones entre términos dentro del diagnóstico que pudieran resultar relevantes para la clasificación.

Para reducir el riesgo de sobreajuste, se añadieron capas de **Dropout (0.3-0.5)** entre las capas densas. Finalmente, se utilizó una capa **Dense** intermedia con activación **ReLU** y una capa de salida **Softmax**, encargada de predecir una de las cuatro clases de estadificación.

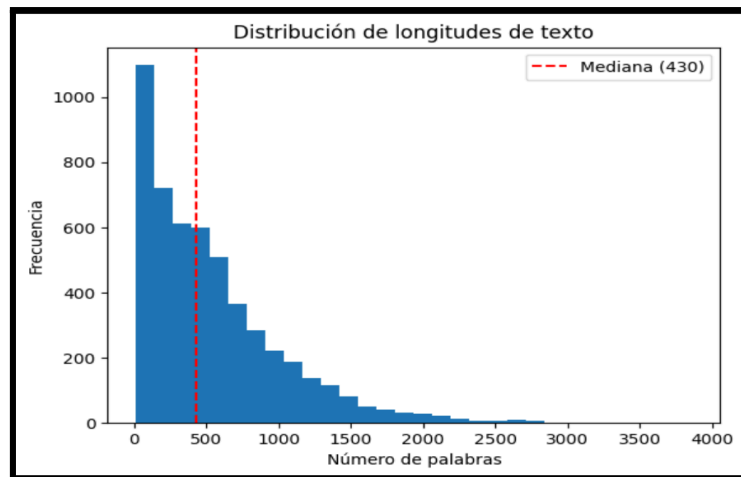
La arquitectura final fue la siguiente:

Embedding → BiLSTM → Dropout → Dense(ReLU) → Dropout → Dense(Softmax)

Experimentos y resultados

Como se podían obtener **embedding distintos por cada token** lo que se hizo fue simplemente hacer la **media de todos los vectores** obtenidos.

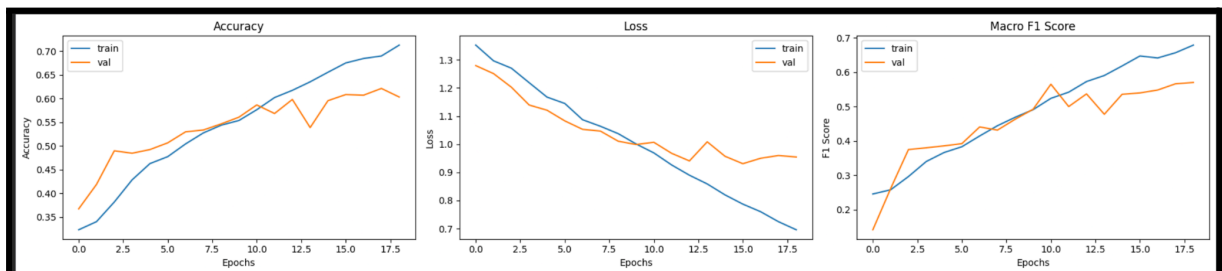
También, cabe mencionar que en este apartado se decidió cambiar el tamaño de la ventana de la media a la mediana que es menor:



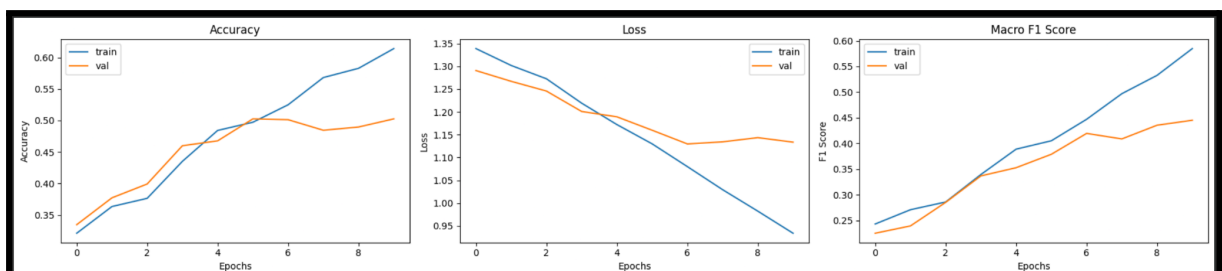
Esto es porque quería probar con una ventana más pequeña para así también se reduzca el número de parámetros a entrenar, ya que como se va a ver se dan casos de overfitting.

Se utilizó el tokenizador propio de **ClinicalBERT**, aplicando *padding* y *truncation* para ajustar todos los textos a la misma longitud. En ambos modelos se utilizó el optimizador **Adam** y la función de pérdida **categorical cross entropy**.

En cuanto al modelo normal sin BiLSTM obtenemos **decentes** resultados:



Pero con BiLSTM resultados peores debido al **sobreaajuste** como causa de que con BiLSTM tenemos más parámetros los cual requiere muchos datos para un buen entrenamiento, cosa que no se da:



Análisis

Los resultados muestran que la arquitectura **sin BiLSTM volvió a funcionar mejor**, obteniendo un **Macro F1 de 0.5439**. Aunque el modelo con BiLSTM mejoraba mucho

durante el entrenamiento, en validación y test no consiguió mantener ese rendimiento, lo que apunta a un posible **sobreajuste**.

Esto puede indicar que **ClinicalBERT ya aporta bastante contexto por sí solo**, por lo que añadir una BiLSTM encima no siempre ayuda. Además, al tratarse de un dataset relativamente pequeño, un modelo más complejo puede acabar ajustándose demasiado a los datos de entrenamiento.

2.6. Propuesta 5: Fine-tuning

ClinicalBERT

En esta última propuesta se decidió realizar **fine-tuning de ClinicalBERT**, es decir, entrenar directamente un modelo preentrenado sobre nuestra tarea de clasificación en lugar de utilizarlo únicamente como extractor de embeddings.

Se eligió **ClinicalBERT** frente a **BioBERT** debido al tipo de datos utilizados en el proyecto. Aunque ambos modelos pertenecen al ámbito biomédico, **BioBERT** está entrenado principalmente sobre artículos científicos y literatura médica (*PubMed*, papers, abstracts, etc.), mientras que **ClinicalBERT** ha sido entrenado sobre **notas clínicas y documentación hospitalaria real**. Dado que los textos del dataset son diagnósticos escritos por médicos, con abreviaturas, términos clínicos y una redacción menos formal que un artículo científico, parecía una opción más adecuada.

Además, **ClinicalBERT está basado en la arquitectura Transformer**, una de las principales diferencias respecto a los modelos probados anteriormente. Mientras que enfoques como **Bag of Words** solo consideran la frecuencia o presencia de palabras, y modelos recurrentes como las **LSTM/BiLSTM** procesan el texto de forma secuencial, los **Transformers pueden analizar todas las palabras del texto al mismo tiempo**, capturando relaciones entre términos aunque estén separados dentro del diagnóstico.

Esto es posible gracias al mecanismo de **self-attention**, que permite al modelo asignar importancia a distintas partes del texto cuando interpreta una palabra. En un contexto médico esto puede resultar especialmente útil, ya que el significado de ciertos términos depende mucho del resto de la frase o de otros conceptos mencionados en el diagnóstico.

Por ejemplo, un término clínico puede adquirir relevancia distinta dependiendo de síntomas, órganos afectados o procedimientos mencionados en el mismo texto. Este tipo de relaciones son difíciles de capturar con representaciones más simples.

Otra ventaja importante es que **ClinicalBERT genera embeddings contextuales**, es decir, una palabra no siempre tiene el mismo vector asociado, sino que su representación cambia según el contexto donde aparece. Esto supone una diferencia importante frente a **FastText**, donde cada palabra tiene una única representación fija independientemente de la frase.

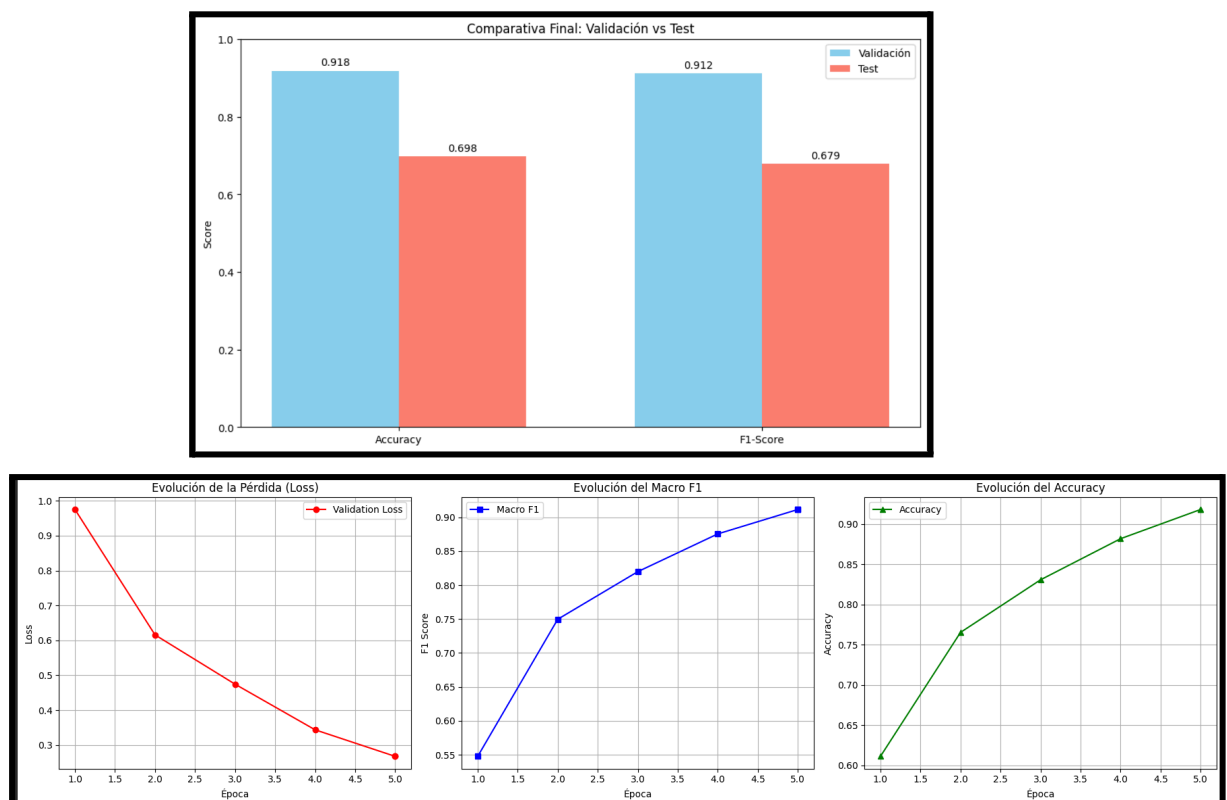
Experimentos y resultados

Se utilizó el tokenizador propio de **ClinicalBERT**, aplicando *padding* y *truncation* construyendo una ventana de **512 tokens**, la razón de que sea 512 es que la media está por encima y 512 es el máximo para el modelo. El dataset se dividió en entrenamiento, validación y test, manteniendo el mismo esquema empleado en propuestas anteriores.

Para el entrenamiento se utilizó la API **Trainer** [10] de la librería **Transformers (Hugging Face)**, lo que permitió gestionar de forma más sencilla el proceso de entrenamiento, validación y evaluación del modelo.

El entrenamiento se realizó durante **5 épocas** porque ya viene fuertemente entrenado y solo queremos ajustarlo a nuestro corpus. Se ha utilizado el optimizador por defecto de Hugging Face (AdamW) y evaluando el modelo al final de cada época.

Durante el entrenamiento se observó una mejora constante tanto en *accuracy* como en *Macro F1*, mientras que la pérdida de validación disminuyó progresivamente.



Análisis

Esta propuesta obtuvo los **mejores resultados de todas las realizadas**, mostrando una mejora muy clara respecto a modelos clásicos, embeddings estáticos y redes neuronales anteriores.

Durante validación, el modelo llegó a un **Macro F1 de 0.91**, lo que indica que ClinicalBERT es capaz de capturar bastante bien el lenguaje clínico presente en los diagnósticos. Esto era esperable, ya que el modelo ha sido entrenado previamente sobre documentación médica real.

Sin embargo, al evaluar en test se produjo una bajada importante del rendimiento (**Macro F1 = 0.68**). Esto sugiere cierto **sobreajuste al conjunto de validación**, probablemente debido al tamaño limitado del dataset y al elevado número de parámetros de ClinicalBERT.

Aun así, incluso con esta caída en test, el modelo sigue obteniendo resultados competitivos y demuestra que el uso de modelos contextuales especializados en lenguaje médico aporta una mejora clara frente a enfoques más tradicionales.

2.7. Modelo Final y predicciones

El modelo final es la propuesta 5. Para realizar las predicciones se ha seguido el mismo procedimiento que el de la Propuesta 5 y se ha entrenado con el corpus de entrenamiento junto con el de evaluación para obtener las predicciones sobre el corpus test.

Esto se encuentra en el notebook **Propuesta5_Final_Predicciones.ipynb**.

3. Conclusiones

El objetivo de este trabajo era clasificar la estadificación del cáncer a partir de diagnósticos médicos en lenguaje natural, comparando distintos enfoques de Procesamiento del Lenguaje Natural para comprobar cuáles se adaptaban mejor al problema.

Como primera aproximación, se probaron modelos clásicos junto con representaciones **Bag of Words**, obteniendo resultados razonables. De hecho, sorprendió que **Regresión Logística con pesado binario** funcionase bastante bien, lo que parece indicar que la presencia de ciertos términos médicos concretos tiene bastante peso a la hora de identificar la estadificación.

Posteriormente, se probaron **embeddings estáticos** mediante **FastText/BioWordVec**, esperando una mejora al capturar algo más de información semántica. Sin embargo, los resultados no superaron claramente a Bag of Words, lo que puede sugerir que, en este problema, el contexto de las palabras es más importante que una representación estática de los términos.

También se exploraron arquitecturas de **Deep Learning**, comparando modelos simples con otros basados en **BiLSTM**. Aunque inicialmente parecía que las arquitecturas más complejas podrían ofrecer mejores resultados, se observó que en varios casos tendían a **sobreajustar**, especialmente debido al tamaño relativamente reducido del dataset y al desbalanceo entre clases.

Finalmente, el mejor enfoque fue el **fine-tuning de ClinicalBERT**, consiguiendo los mejores resultados globales. Esto era relativamente esperable, ya que se trata de un modelo basado en **Transformers**, capaz de tener en cuenta el contexto completo del texto, y además entrenado previamente sobre documentación clínica real, algo muy parecido al tipo de diagnósticos utilizados en este proyecto.

Aun así, también se observó cierta diferencia entre validación y test, lo que indica que todavía existe margen de mejora en la capacidad de generalización del modelo. Como posibles líneas futuras, podría resultar interesante probar técnicas para reducir el desbalanceo de clases (por ejemplo, *data augmentation*), realizar una búsqueda más exhaustiva de hiperparámetros o experimentar con otros modelos biomédicos similares.

En general, este trabajo ha permitido comprobar cómo el tipo de representación textual y la arquitectura elegida pueden influir mucho en el rendimiento final, y cómo en un dominio tan específico como el médico el uso de modelos especializados puede marcar una diferencia importante.

Referencias

- [1] [Spacy](https://spacy.io/) : <https://spacy.io/>
- [2] [CountVectorizer \(Pesado Binario\)](https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.CountVectorizer.html) : https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.CountVectorizer.html
- [3] [TF-IDF](https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.TfidfVectorizer.html) : https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.TfidfVectorizer.html
- [4] [SVM](https://scikit-learn.org/stable/modules/svm.html) (Doc) : <https://scikit-learn.org/stable/modules/svm.html>
- [5] [Logistic Regression](https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html) (Doc) : https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html
- [6] [Naive Bayes](https://scikit-learn.org/stable/modules/naive_bayes.html) (Doc) : https://scikit-learn.org/stable/modules/naive_bayes.html
- [7] [en_core_sci_lg](https://allenai.github.io/scispacy/) : <https://allenai.github.io/scispacy/>
- [8] [BioWordVec](https://github.com/ncbi-nlp/BioWordVec.git) (Github) : <https://github.com/ncbi-nlp/BioWordVec.git>
- [9] [ClinicalBert](https://github.com/EmilyAlsentzer/clinicalBERT.git) (Github) : <https://github.com/EmilyAlsentzer/clinicalBERT.git>
- [10] [Trainer](https://huggingface.co/docs/transformers/es/trainer) (Doc) : <https://huggingface.co/docs/transformers/es/trainer>
- [11] [BioBert](https://github.com/dmis-lab/biobert.git) : <https://github.com/dmis-lab/biobert.git>